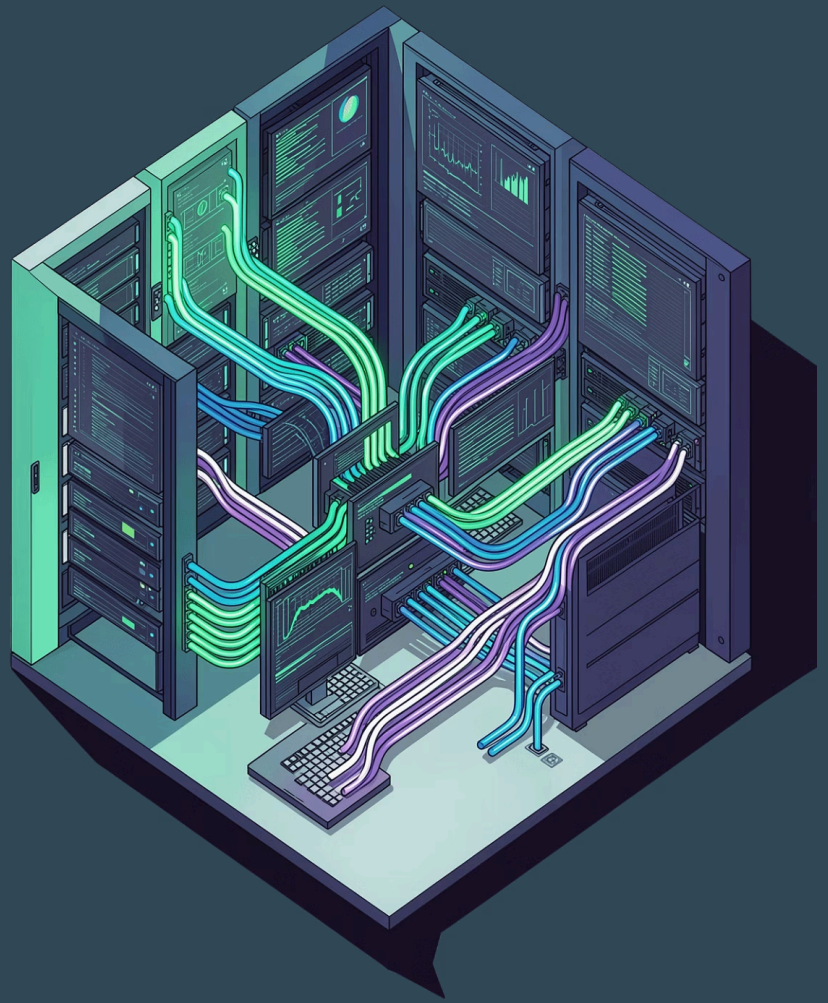


Bastionado de Servidores Web: Ocultamiento de Banners y Seguridad en Cabeceras HTTP

Auditoría y Hardening de Servicios Web en Entornos Contenedorizados

Mg. Gabriel Guerrero Arellano

ASIGNATURA	FECHA	INSTITUCIÓN
Sistemas de Comunicación y Seguridad Informática	Septiembre 2026	Universidad Cooperativa de Colombia

De la Teoría y el Reconocimiento a la Ingeniería Defensiva

Recuento de Aprendizajes (Semanas 1 a 4) y Hilo Conductor

RECORRIDO DEL CURSO — SEMANAS 1 A 4

01

Semana 1 — Fundamentos de Seguridad

Tríada CID (Confidencialidad, Integridad y Disponibilidad) y Principio de Defensa en Profundidad.

02

Semana 2 — Reconocimiento y Escaneo

Identificación de puertos abiertos y mapeo de servicios en la red.

03

Semana 3 — Gestión de Vulnerabilidades

Evaluación del riesgo por exposición de servicios e identificación de fallas conocidas (CVE).

04

Semana 4 — Gestión de Identidades y Accesos

Filosofia ZERO TRUST, ZERO privilegios y IAM seguridad robusta.

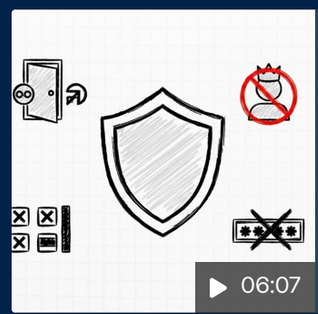
🎯 CONEXIÓN CON LA SEMANA 5 (HOY)

Fase Defensiva Activa: Bastionado

Habiendo aprendido cómo los atacantes mapean servicios e identifican vulnerabilidades (Semanas 2–3) y cómo desplegarlos sobre Docker (Semana 4), hoy pasamos a la fase defensiva activa: aplicar bastionado (hardening) sobre nuestro servidor Nginx para eliminar la fuga de información y proteger a los usuarios.

El conocimiento del atacante es el punto de partida del defensor efectivo.

VIDEO BASICO DE QUE ES HARDENING:



YouTube

Hardening de Servidores: Có...

¿Es tu servidor una fortaleza o un libro abierto para los atacantes?...

Contexto de Seguridad en Servidores Web

La Capa de Aplicación como Superficie de Ataque

🌐 Cara Pública de la Infraestructura

Los servidores HTTP/HTTPS son los activos más expuestos a escaneos e intentos de explotación masivos en Internet.

⚠️ El Riesgo de las Configuraciones por Defecto

Al instalar un servidor web, el fabricante incluye banners informativos y omite restricciones estrictas para garantizar compatibilidad, no seguridad.

🛡️ Estrategia de Bastionado

Transformar un servicio 'funcional pero expuesto' en un servicio 'bastionado y resiliente' mediante configuración activa.

Fuga de Información y Banners de Versión

Marco Teórico I — El Ciclo de Reconocimiento (Footprinting)

¿Qué es el Banner Grabbing?

Banner Grabbing

Técnica en la que un cliente obtiene la firma y versión exacta del software (`Server: nginx/1.25.3`) enviada por el servidor web.

Búsqueda de CVEs

Al revelar la versión exacta, un atacante automatiza la búsqueda de vulnerabilidades públicas para lanzar ataques dirigidos.

Solución Defensiva

DIRECTIVA DE NGINX

```
server_tokens off;
```

Ocultar la versión del servidor entorpece el reconocimiento automatizado y eleva el costo del ataque para el adversario.

⚠ Sin esta directiva, la cabecera `Server: nginx/1.25.3` es visible para cualquier cliente HTTP, incluyendo escáneres automatizados.

OWASP y el OWASP Top 10: El Estándar Global de Seguridad Web



FUNDAMENTOS Y FILOSOFÍA DE OWASP

ORGANIZACIÓN MUNDIAL, NEUTRAL E INDEPENDIENTE

Open Worldwide Application Security Project (OWASP) es una fundación global dedicada a majorar la seguridad del software a nivel internacional sin intereses comerciales.

RECURSOS 100% LIBRES Y ABIERTOS

Toda la documentación, herramientas y metodologías de OWASP están disponibles de forma totalmente gratuita para la comunidad tecnológica mundial.

COMUNIDAD COLABORATIVA GLOBAL

El ecosistema de OWASP se sostiene mediante el trabajo conjunto de miles de expertos e investigadores de seguridad a nivel internacional.

EL OWASP TOP 10: MÉTRICAS, METODOLOGÍA Y RIESGOS



ESTÁNDAR GLOBAL DE CONCIENTIZACIÓN

Documento de referencia que clasifica los 10 riesgos de seguridad más destructivos y frecuentes que afectan a las aplicaciones web.

METODOLOGÍA BASADA EN EVIDENCIA REAL

Se elabora mediante el análisis estadístico masivo de vulnerabilidades reales reportadas por empreses, auditores y firmas de ciberseguridad.

TERMÓMETRO DE AMENAZAS EMERGENTES

Sirve como métrica prioritaria para identificar vectores de araque críticos e identificar patrones de vulnerabilidades en evolución.



A05:2021 – MALA CONFIGURACIÓN DE SEGURIDAD

CONEXIÓN CON CLASE: A05:2021

Ubicada en el puesto #5 del ranking mundial, demuestra cómo puertos innecesarios abiertos, credenciales por defecto y cabeceras desconfiguradas exponen un sistema por falta de hardening.

APLICACIÓN PRÁCTICA E IMPACTO EN LA INDUSTRIA

GUÍA DE BUENAS PRÁCTICAS PARA DEVS Y SYSADMINS

Enseña a desarrolladores y administradores de sistemas los errores de arquitectura e infraestructura que deben evitarse al desplegar servidores.

MARCO DE AUDITORÍA PREVIO A PRODUCCIÓN

Estándar utilizado por las empresas para realizar pruebas de penetración y evaluación de riesgos antes de lanzar un software al mercado.

CICLO DE MITIGACIÓN Y PREVENCIÓN



CONFIGURACIÓN SEGURA DESDE EL DISEÑO (HARDENING)

Deshabilitar servicios innecesarios, cambiar valores predeterminados y asegurar cada componente de la infraestructura.

APLICACIÓN DE PARCHES Y ACTUALIZACIONES CONTINUAS

Mantener todos los componentes del sistema, librerias y servidores actualizados contra vulnerabilidades conocidas.



PRINCIPIO DE MENOR PRIVILEGIO Y AUDITORÍAS

Restringir el acceso a usuarios y procesos solo a lo estrictamente necesario, validando continuamente el estado de seguridad.

Gemini Notebook

A05:2021 – Mala Configuración de Seguridad (Security Misconfiguration)

Marco Teórico II — OWASP Top 10

POSICIÓN EN EL RANKING OWASP

#5

Entre las amenazas web más críticas a nivel mundial según OWASP Top 10 2021.

CONSECUENCIA

Facilita la exploración del sistema y habilita vectores de ataque en la capa del cliente.

Causas Frecuentes

Exposición de Detalles del Sistema

Cabeceras HTTP revelan versión exacta del servidor, sistema operativo y tecnologías usadas.

Ausencia de Políticas Defensivas

Falta de cabeceras de protección para el navegador del cliente (`X-Frame-Options`, `X-Content-Type-Options`).

Configuraciones por Defecto

Archivos de configuración instalados por el fabricante sin revisión ni ajuste de seguridad.

Vectores de Ataque: Clickjacking y MIME-Sniffing

Marco Teórico III — Amenazas Clave en el Cliente

1. CLICKJACKING (UI REDRESS ATTACK)

Ataque donde una web maliciosa incrusta nuestra aplicación dentro de un `<iframe>` invisible para engañar al usuario y hacer que presione botones críticos sin su consentimiento.

CABECERA DEFENSIVA

```
X-Frame-Options: SAMEORIGIN
```

Prohíbe que sitios externos encuadren nuestra web en un iframe.

2. MIME-SNIFFING

Comportamiento donde el navegador ignora el tipo de archivo declarado por el servidor e intenta "adivinarlo", pudiendo interpretar archivos estáticos (ej. imágenes) como scripts ejecutables.

CABECERA DEFENSIVA

```
X-Content-Type-Options: nosniff
```

Obliga al navegador a respetar estrictamente el tipo de contenido enviado, anulando el MIME-sniffing.

Cabeceras HTTP de Seguridad

Marco Teórico IV — Mecanismos de Defensa HTTP — Las Directivas de Protección en Nginx

Directiva	Valor	Protección que Otorga
`server_tokens off;`	off	Suprime el número de versión en la cabecera Server, dificultando el reconocimiento automatizado.
`X-Frame-Options`	SAMEORIGIN	Prohíbe que sitios externos encuadren nuestra web en un `<iframe>`, bloqueando Clickjacking.
`X-Content-Type-Options`	nosniff	Obliga al navegador a respetar el tipo de contenido declarado, anulando el MIME-sniffing.

Declaración en `nginx.conf`:

```
server_tokens off;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
```

ⓘ La directiva `always` garantiza que las cabeceras se envíen incluso en respuestas de error (4xx, 5xx).

Metas Técnicas del Taller Práctico

Objetivos y Alcance del Laboratorio

OBJETIVO GENERAL

Desplegar, auditar y aplicar hardening a un servidor Nginx en Docker para mitigar la fuga de información y proteger la navegación.

METAS ESPECÍFICAS

01

Orquestación con Docker Compose

Configuración de `docker-compose.yml` con mapeo al puerto 8085.

02

Bastionado de Nginx

Aplicación de directivas de seguridad en `nginx.conf`: `server_tokens`, `X-Frame-Options`, `X-Content-Type-Options`.

03

Auditoría por Consola

Verificación e inspección de cabeceras HTTP mediante `curl.exe` en PowerShell local.

Entorno de Laboratorio Contenedorizado

Arquitectura de la Infraestructura

Componentes del Entorno

Componente	Detalle
Host	Windows 11 / PowerShell
Motor de Contenedores	Docker Desktop
Contenedor Base	Nginx Alpine (nginx:alpine)
Nombre de Contenedor	nginx_seguro
Mapeo de Puertos	8085 (Host) → 80 (Contenedor)
Herramienta de Auditoría	Cliente cURL nativo (curl.exe)

Diagrama de Flujo

1. **PowerShell (Host)**
   Ejecuta comandos docker y curl.exe
2. **Docker Desktop**
   Orquesta el contenedor nginx_seguro
3. **Nginx Alpine :80**
   Sirve contenido con cabeceras bastionadas
4. **Puerto 8085**
   Punto de acceso y auditoría desde el host

Paso 1: Configuración de docker-compose.yml

Laboratorio Paso 1 — Infraestructura en Docker

CÓDIGO FUENTE — DOCKER-COMPOSE.YML (YAML)

```yaml
services:
  web-target:
    image: nginx:alpine
    container_name: nginx_seguro
    ports:
      - "8085:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
```

Explicación del Archivo

image: nginx:alpine

Imagen oficial de Nginx sobre Alpine Linux — mínima, segura y liviana.

ports: 8085:80

Mapea el puerto local 8085 del host hacia el puerto 80 interno del contenedor.

volumes: ./nginx.conf

Enlaza el archivo de configuración personalizado nginx.conf al contenedor, reemplazando la configuración por defecto.

Paso 2: Definición de Reglas en nginx.conf

Laboratorio Paso 2 — Aplicación de Bastionado

CÓDIGO FUENTE — NGINX.CONF (NGINX)

```
server {
    listen 80;
    server_name localhost;

    # 1. Ocultar la versión exacta de Nginx
    server_tokens off;

    # 2. Inyección de cabeceras defensivas
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

Análisis Línea a Línea

server_tokens off;

Elimina la versión de Nginx de la cabecera Server. Antes: `nginx/1.25.3` → Después: `nginx`

X-Frame-Options: SAMEORIGIN

Bloquea el Clickjacking impidiendo que la página sea incrustada en iframes de dominios externos.

X-Content-Type-Options: nosniff

Fuerza al navegador a respetar el Content-Type declarado, previniendo el MIME-sniffing.

Paso 3: Despliegue y Administración en Vivo

Laboratorio Paso 3 — Ejecución y Recarga del Servicio

1. DESPLIEGUE DEL CONTENEDOR

```
docker compose up -d --force-recreate
```

Levanta el contenedor en segundo plano y fuerza la recreación si ya existía.

2. VERIFICACIÓN DE ESTADO ACTIVO

```
docker ps
```

Confirma que el contenedor nginx_seguro está en estado 'Up' y escuchando en el puerto 8085.

3. RECARGA SIN CAÍDA DE SERVICIO

```
docker exec nginx_seguro nginx -s reload
```

Recarga la configuración de Nginx sin interrumpir el servicio ni las conexiones activas.

ⓘ El flag `--force-recreate` garantiza que los cambios en `nginx.conf` sean aplicados incluso si el contenedor ya existía previamente.

Paso 4: Inspección de Cabeceras HTTP

Laboratorio Paso 4 — Auditoría de Seguridad por Consola

CURL.EXE es una herramienta de consola (CLI) utilizada para transferir datos y hacer peticiones a servidores mediante protocolos como HTTP, HTTPS o FTP, sin necesidad de usar un navegador web.

Diferenciación Técnica Importante

curl en PowerShell

En PowerShell, el comando `curl` es un alias de `Invoke-WebRequest` y NO es el cliente cURL real.

curl.exe (Nativo)

Se debe invocar `curl.exe` explícitamente para utilizar el cliente nativo de cURL con todas sus opciones HTTP.

COMANDO DE AUDITORÍA EJECUTADO (POWERSHELL)

```
curl.exe -I http://localhost:8085
```

El parámetro `-I`: Le pide al servidor web que devuelva **únicamente las cabeceras de respuesta HTTP** (los metadatos) sin descargar todo el contenido del sitio web (HTML, imágenes, scripts). Es la herramienta preferida por los auditores de seguridad para inspeccionar cómo está configurado un servidor.

⚠ Usar `curl` sin la extensión `.exe` en PowerShell retornará un objeto .NET, no la salida de texto esperada.

Paso 5: Resultados de la Ejecución en Consola

Laboratorio Paso 5 — Análisis de Evidencias Reales

RESPUESTA HTTP OBTENIDA (EVIDENCIA REAL)

```
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 29 Sep 2026 13:28:56 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Tue, 15 Sep 2026 14:18:52 GMT
Connection: keep-alive
ETag: "6aa953cc-380"
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Accept-Ranges: bytes
```

Análisis de Resultados

Elemento Auditado	Valor Obtenido	Estado de Seguridad
Cabecera Server	Server: nginx	✅ Exitoso: Versión exacta removida.
X-Frame-Options	SAMEORIGIN	✅ Exitoso: Protección contra Clickjacking activa.
X-Content-Type	nosniff	✅ Exitoso: Protección contra MIME-Sniffing activa.

Cierre Técnico y Conexión con la Próxima Clase

Conclusiones y Próximos Pasos — Semana 6

✅ CONCLUSIONES DE LA SESIÓN

En este taller estamos realizando una práctica de **Bastionado de Servidores Web (*Web Server Hardening*)** utilizando contenedores Docker.

El objetivo principal es transformar un servidor web que viene configurado por defecto (e inseguro) en un servidor **resistente a fases de reconocimiento y ataques comunes de la capa de aplicación**.

El laboratorio se divide en tres momentos clave:

1. **Despliegue Aislado:** Levantamos un servidor Nginx en un entorno controlado sobre Docker (`docker-compose.yml`), simulando la puesta en producción de un servicio web.
2. **Aplicación de Políticas de Seguridad (*Hardening* en `nginx.conf`):**
   - `server_tokens off;` **(Mitigación de Fuga de Información):** Ocultamos la versión exacta de Nginx. Esto impide que atacantes usen escáneres automáticos para identificar la versión y buscar vulnerabilidades conocidas (CVEs) para explotarla.
   - `X-Frame-Options: SAMEORIGIN` **(Protección Anti-Clickjacking):** Prohibimos que páginas externas incrusten nuestro sitio en un `<iframe>` invisible para engañar a los usuarios.
   - `X-Content-Type-Options: nosniff` **(Protección Anti-MIME Sniffing / XSS):** Obligamos al navegador del cliente a respetar el tipo de archivo enviado, evitando que ejecute código malicioso camuflado en imágenes o archivos estáticos.
3. **Auditoría y Verificación:** Usamos `curl.exe -I` como herramientas de auditoría (*petición de inspección*) para comprobar en tiempo real que el servidor efectivamente está entregando las cabeceras defensivas requeridas.

➡️ SOON CONEXIÓN CON LA SEMANA 6

Próxima Clase: HTTPS / TLS / SSL

El Problema Actual

La transmisión HTTP actual (puerto 8085) viaja en texto plano y es susceptible a escuchas pasivas o ataques Man-in-the-Middle (MitM).

Módulo Semana 6

Implementación de cifrado en tránsito mediante HTTPS / TLS / SSL, generación de certificados y activación de la cabecera HSTS (HTTP Strict Transport Security).